Text as Data for Economic Research
Transformers and LLMs as Measurement Tools

Class 4. From Pipelines to Validated Annotations

June 10, 2026

Tiziano Rotesi

University of Turin

Where This Sits in the Course

- **Class 1**: we built measures by hand — dictionaries, counts, PMI.
- **Class 2**: we *learned* (supervised) or *discovered* (unsupervised) categories from features.
- **Class 3**: we changed the representation — a geometry of meaning, but *one static vector per word*.
- **Today**: representations that depend on *context*, and the off-the-shelf models that produce them.

Throughout: a model's output is a measurement. The recurring question is whether it measures what we claim.

The Goal Has Not Changed

We still want a **variable** we can put in a regression: a sentiment score, a topic indicator, an attribute rating.

- Classes 1–3: we engineered or learned that variable from counts and vectors.
- Today: a pretrained model can often hand us the variable directly.
- New power, same discipline: a generated variable carries measurement error, and must be validated.

From Static to Contextual

Recall: Word2Vec Gives One Vector per Word

In Class 3 each word type got a *single* dense vector, learned from its average context across the corpus.

- Great for similarity, analogies, dictionary expansion.

- But a word's meaning is collapsed into *one* point, regardless of the sentence it appears in.

The Polysemy Problem

One vector cannot serve a word with several senses:

- "*plot*": narrative, conspiracy, plot of land, a scatter plot.
- "river **bank**" vs. "central **bank**" — same vector in Word2Vec.

For an economist this is a validity threat: a sentiment or topic measure that cannot tell "*bank*" apart will misread financial text.

What We Want: Meaning in Context

A representation of a token that **depends on the whole sentence**:

- "central **bank**" $\Rightarrow$ a vector near *policy, rates, inflation*.
- "river **bank**" $\Rightarrow$ a vector near *water, shore*.

This is exactly what transformers produce: **contextual embeddings**.

The Engine Underneath: Language Modeling

A **(causal) language model** is trained to predict the next token from all the tokens before it — left to right.

- **The task**: given "*The central bank raised . . .*", predict "*rates*".
- **The labels**: the next word in the text itself — no human annotation needed (*self-supervised*).
- **The scale**: hundreds of billions of such predictions across web-sized corpora (news, books, Wikipedia, . . .).
- By solving this prediction task at scale, the model learns word meaning, syntax, and world knowledge as a byproduct — not as an explicit goal.

Connection to Class 1

- This resembles the **multinomial language model** of Class 1: a model of the process generating documents.

- Difference: there it was *bag of words*, order-free. Here the **sequence** is everything, and we frame it as a prediction problem.

A bag-of-words model fails here because it discards order entirely. We need a representation that can capture long-range dependencies across a sequence.

(Intro to) Transformers

Transformers

- New architecture introduced in 2017 — Vaswani et al., "Attention Is All You Need."
- Stacks of **self-attention** layers.
- Goal: a **contextualized representation for each token at each position**.
- Parallelizable $\Rightarrow$ trainable at enormous scale.

We will *use* pretrained transformers, not train them. But we need enough of the mechanism to know what they can and cannot measure.

Two Key Innovations

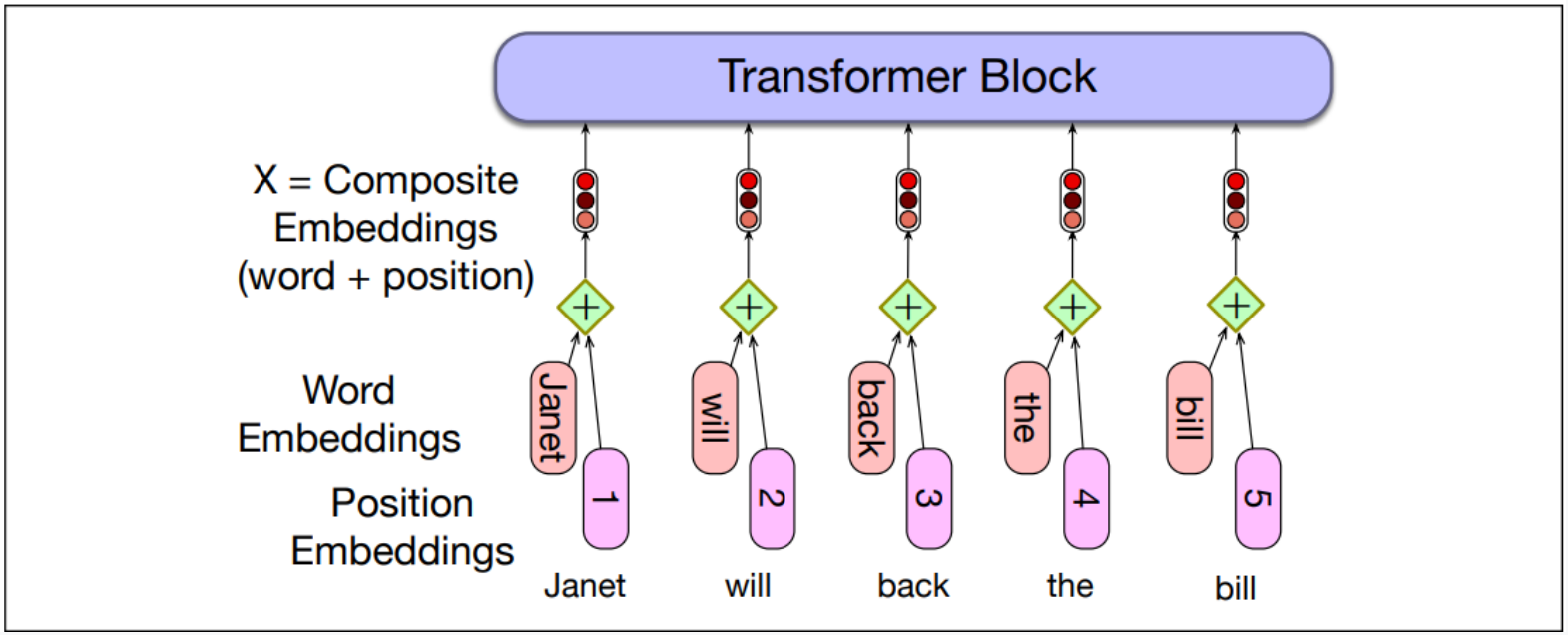
1. **Positional encoding** — the model has no inherent sense of order, so position is added to each token's embedding.

2. **Self-attention** — each token's representation is built from the tokens it "attends to" in the sequence.

The Input: Token + Positional Embeddings

- **Token embedding**: a vector of dimension d (as in Class 3) — semantic, but position-blind.
- **Positional embedding**: encodes *where* the token sits.
- The two are **added**, giving each token a representation that reflects both meaning and place.

The Input, Visually



Jurafsky & Martin, *Speech and Language Processing*.

Self-Attention: the Idea

Self-attention lets the model **prioritize the relevant tokens** for each position. Examples:

- "The **keys** to the cabinet **are** on the table." — subject–verb agreement across distance.
- "The chicken crossed the road because **it** . . ." — pronoun–antecedent.
- ". . . along the **pond** . . . the **bank** . . . the **water**." — disambiguating *bank*.

Self-Attention: Notation

Turn initial token embeddings x_i into contextual embeddings a_i by letting tokens interact:

$$a_i = \sum_j \alpha_{ij} x_j, \qquad \alpha_{ij} = f(x_i, x_j)$$

- a_i is a **weighted average** of all tokens' representations.
- The weights α_{ij} — how much token i attends to token j — are nonnegative, sum to one, and **learned from data**: the model trains them so that attending to the right tokens helps predict the next word.

Training: Self-Supervision at Scale

- At each position, the model makes a prediction, which is scored against the **true next token** in the text.
- **Self-supervised**: the labels are just the next words — no human annotation needed, so training scales to web-sized corpora.
- The model adjusts its attention weights to improve next-token prediction — and in doing so learns contextual meaning.

Parameter counts are huge: GPT-3 had ~175 billion.

Data: Web-Scale Corpora

- Most common source: **Common Crawl** — web snapshots (news, books, Wikipedia, . . .).
- Cleaned derivatives (e.g. C4, ~750 GB): deduplicated, filtered.
- GPT-3: hundreds of billions of tokens from web, books, Wikipedia.

Implication for research: the training data may overlap with *your* sample, or post-date your event — a contamination/look-ahead risk we return to.

Two Families: GPT vs. BERT

- **GPT** (decoder, *causal*): predict the next token, left to right. Good at generation and prompting.

- **BERT** (encoder, *bidirectional*): predict masked tokens using left *and* right context. Good at producing features to classify.

Both give **contextual embeddings**; they differ in how we put them to work.

Contextual Embeddings, the Payoff

- The final-layer vector z_i is the token’s **meaning in context**.
- Unlike static Word2Vec, the same word gets different vectors in different sentences.

From Coenen et al. (2019).

From Language Model to Measurement

The Pivot

We do not care about generation for its own sake. We want a **measure**: a clean variable for each document — a sentiment score, a topic indicator, a policy stance.

A pretrained model can produce that variable in three ways, differing on a single axis: **how much task-specific labeled data they require**.

Three Routes to a Measurement

1. **Embeddings as features** — extract contextual vectors and feed them into your own classifier or regression. The pretrained model is not modified.

2. **Fine-tuning** — add a task head and train on your labeled data. The pretrained weights adapt to your domain and task.

3. **Zero-shot / prompting** — ask the model directly in natural language. No labeled training data — the model uses its existing language knowledge.

Increasing flexibility, decreasing need for labeled data — and increasing need for validation.

Route 1: Embeddings as Features

The most direct bridge from Class 3: pass each document through a pretrained encoder and use the output **vector** as your feature.

- In practice: load a model (e.g. `bert-base-uncased`), run each document through it, and take the average of the output token vectors — one dense vector per document.
- Feed those vectors into a logistic regression or any classifier from Class 2. The pretrained model is not retrained.
- Unlike Word2Vec, the representation depends on context: "bank" in "central bank" and "river bank" produce different vectors.

Advantage: no fine-tuning required. Limit: the downstream classifier still needs labels.

Route 2: Fine-Tuning

- Take a pretrained encoder (e.g. BERT) and add a small classification layer on top — it maps the document's output vector to your label.

- Train on *your* labeled examples. The base model already "knows language"; you only teach it your specific task.

- The result is a **frozen, reproducible model** you can then apply to millions of documents.

Best when you have a few thousand labels and a specific domain.

Worked Case: ClimateBERT

Bingler, Kraus, Leippold & Webersinke (2022)

- Fine-tune BERT on **17,300+ labeled sentences** from annual and sustainability reports. Labels follow the Task Force on Climate-related Financial Disclosures (TCFD) framework: governance, strategy, risk management, and metrics & targets.
- Apply to 818 companies that publicly endorsed TCFD, 2014–2019.
- Domain match matters: a model tuned on the right register reads financial disclosure text far better than a generic sentiment classifier.

ClimateBERT: What They Find

TCFD support raises climate-disclosure content by only **2.2 percentage points** — and the increase is not where it matters.

- **Grows**: governance and risk management (process, optics).
- **Barely moves**: strategy and metrics & targets — the categories investors and supervisors need to assess actual climate exposures.

"**Cheap talk and cherry-picking**": firms largely repackage existing content rather than commit to new disclosures.

Validation: outperforms keyword methods. France's mandatory disclosure regime (Article 173) acts as a natural experiment — higher and more balanced disclosure under mandate confirms the measure captures real variation.

Worked Case: Hansen et al. (2023)

"Remote Work Across Jobs, Companies, and Space," NBER WP 31007

- Fine-tune DistilBERT on **30,000 human-labeled job-posting sequences** to detect remote-work offers — the WHAM (Work from Home Algorithmic Measure) model.
- Applied to **250+ million postings** across 5 English-speaking countries, 2014–2023. Achieves 99% accuracy vs. dictionary methods.
- **Finding**: US remote-work share rose from ~4% (2019) to 13%+ (2023); highly uneven across occupations, firms, and cities.

Same logic as ClimateBERT: labeled data → domain classifier → measurement at scale.

Route 3: Zero-Shot and Prompting

No labeled training data. The model uses its existing language knowledge directly.

- **NLI zero-shot**: supply candidate labels as natural-language hypotheses. "This text is about `<monetary policy>`." The model scores how much the text *entails* each.

- **Completion / prompting**: "The sentiment of this passage is: ___." Compare $P(\text{"positive"} \mid \text{prompt})$ vs. $P(\text{"negative"} \mid \text{prompt})$ — pick the larger.

Both exploit the same insight: the model already knows what words mean in context — you *ask* it, rather than retrain it.

Pipelines: Measurement Off the Shelf

A HuggingFace `pipeline` wraps **tokenizer** + **model** + **post-processing**:

- `sentiment-analysis` — label + score.
- `zero-shot-classification` — your labels, no training.

Two lines of code turn a corpus into a column of measurements.
Today's notebook does exactly this.

Caution: Domain Matters

A model trained on movie reviews can misread financial or clinical text, where the same words carry different valence (“*liability*”, “*aggressive*”).

Always check the training domain is close to yours — a recurring validity threat, and the first reason to validate.

LLMs as Annotators

Prompting in Place of Hand-Coding

This is Route 3, taken seriously as a **research method**.

Instead of a fixed classifier, **prompt a general LLM** with your labeling rule in plain language — the model plays the role of a human coder.

- Can rival — or beat — crowd workers on many annotation tasks.
- Deliverable: a column of labels, one per document, at research scale.

Gilardi, Alizadeh & Kubli (2023), "ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks."

The Measurement Paradigm

Asirvatham, Mokski & Shleifer (2026), "GPT as a Measurement Tool"

Most data humans produce is **qualitative** (speeches, filings, curricula, interviews): rich but not testable.

- An LLM can *read* qualitative data and *rate* an attribute, like a human coder.
- If you can describe an attribute in words — how "pro-innovation" or "populist" a passage is — you can ask for it consistently across thousands of documents.

What LLM Annotators Can Do

Describe the attribute in plain language and the model applies the rule consistently across thousands of documents. The five core annotation tasks:

- **Rate** on a scale (e.g. 0–100): how populist, how optimistic, how hawkish a passage is.
- **Classify**: assign a category from your predefined label set.
- **Rank** (pairwise): which of two passages better fits the attribute?
- **Extract**: pull out specific information — entities, dates, figures.
- **Discover**: identify in natural language what distinguishes two classes of documents — the model writes a data-driven codebook.

Annotation Cost at Scale

Per-corpus cost of rating texts on ten attributes (Asirvatham et al., Table 1):

	gpt-5-nano	gpt-5	human
240 State-of-the-Union speeches	$0.14	$3.46	~$2,600
100k full-text sermons	$43	$1,083	~$700,000

Roughly **700× to 17,500× cheaper** than crowdsourcing — tasks that were previously too expensive to contemplate become feasible.

A General Validation Framework

The paper proposes two criteria that *any* LLM-generated measure should satisfy — not specific to GPT or to any tool:

- **Accuracy**: the model's label agrees with a trusted external benchmark — human coding for subjective constructs, objective outcomes where available.

- **Directness**: the label is driven by the text's *content*, not by memorized training data (*contamination*) or correlated shortcuts (*shortcut inference*).

These map onto the same threats we discussed in Class 2 for any learned measure.

What the Evidence Shows

Tested across **1,000+ distinct annotation tasks** from real empirical research, spanning diverse domains and construct types:

- **Accuracy**: median agreement with human labels ≈ 0.85. GPT outperforms individual human raters on 13 of 23 tested variables.
- **Contamination**: no performance decay on post-training-cutoff data — the model reads the text, it does not recall it.
- **Shortcut inference**: removing the intended signal attenuates ratings by ~81% in real data — the label tracks the target construct, not a correlated proxy.

Application: Technology Adoption History

Construct: adoption lag — years from first working prototype to widespread use by the target audience.

Source: 18 million Wikipedia article titles, filtered through a six-stage LLM pipeline to **37,000 historically significant technologies**. For each: year of invention, year of adoption, and 21 attributes (financing, network effects, supply chain complexity, . . .).

Main finding: adoption lags fell tenfold — from ~50 years in the 19th century to ~5 years today.

- **Fastest**: military technology (−27% relative to contemporaries).
- **Slowest**: energy, medical, agricultural (20–40% longer than average).

Writing the Prompt: a Checklist

The prompt *is* your **measurement instrument**:

1. **State the task**: unit of analysis, what to judge, how to handle edge cases.
2. **Define the attribute**: an explicit definition — particularly important for ambiguous constructs.
3. **Define the label set or scale**: exhaustive and exclusive; anchor the endpoints.
4. **Request structured output** (JSON): same schema every call, machine-parseable.
5. **Instruct independent judgment**: "Rate each attribute separately — do not infer one from another."
6. **Ask for evidence**: a verbatim quote and a one-line reason. The model produces better labels *and* you get a fast audit trail — spot-check whether the label reflects the content, not a

How Much Does Prompt Wording Matter?

A key finding from the paper: **simple prompts perform nearly as well as elaborate ones**.

- Across 10 attributes, alternative phrasings correlated 0.92 with the detailed baseline prompt on average.
- Exception: **ambiguous constructs** are more sensitive to definition wording (correlation drops to ~0.64 for concepts like "anger").

Implication: invest in **clarity of the construct**, not in prompt engineering. The definition matters — the exact phrasing usually does not.

Prompt Hygiene

Wording, examples, and the label set can all move the output. Standard practice:

- Fix the prompt before running at scale. Set `temperature = 0` for reproducibility.
- Log the exact model version and date — model behavior can change across versions.
- Pilot on a small sample first. Hold out a test set before finalizing the prompt, to guard against p-hacking on the labeling side.

Live: worked example in today's notebook.

A Note on Tools

Asirvatham et al. release **GABRIEL**, a Python package that wraps these operations and handles parallelization and output parsing.

The paper is explicit: "GABRIEL does nothing that couldn't be done through GPT alone. This paper's aim is to validate LLMs generally in measuring qualitative data, not GABRIEL specifically."

The transparency concern: any package adds a layer between you and what the model is actually asked. If you use such a tool, read the prompts it sends — they are your measurement instrument, and you need to be able to describe and defend them.

Validating Model Labels *(the core)*

Why You Cannot Skip This

LLM labels are appealing: fluent, plausible, and delivered instantly. But:

- **Black box**: billions of parameters, opaque (or proprietary).
- **Sycophancy**: models answer even when the instruction is ambiguous.
- **Hallucination**: confident, wrong, and hard to predict.

A plausible label is not a correct measure. Never feed model labels into a regression unchecked.

Two Validation Paradigms

We sequence by the assumptions they require:

1. **Benchmark-based**: compare model labels to *trusted human labels*. (Assumes such labels exist and are reliable.)

2. **Benchmark-free**: validate the labels using the model itself, when human labels are unavailable or themselves unreliable (Hansen 2026).

Paradigm 1: Human Benchmark

The standard design:

1. Draw a sample with **trusted human labels**.
2. Generate model labels for the same items.
3. Compare and **quantify the error**.
4. Translate it into measurement error for the downstream estimate (Class 5).

Agreement Metrics

- **Accuracy** and the **confusion matrix** — where do errors fall?
- **Cohen's** κ — agreement corrected for chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

  with p_o observed and p_e chance agreement.

- **Correlation / rank correlation** for continuous ratings.

Read the Disagreements

The most informative output is *where the model and humans disagree*.

- Disagreements reveal which construct the model is *actually* measuring.

- Systematic disagreement (e.g. sarcasm always read as positive) is a **bias**, not noise — and biases your coefficients.

But Is the Human Benchmark Reliable?

The motivation for Hansen (2026)

Human labels are not ground truth:

- Subjectivity — and drift as the coder learns or tires.
- Crowd workers increasingly **use LLMs themselves** (estimates: a third or more).
- Genuine experts are slow and expensive.

So: how do we validate when a reliable external benchmark is **unavailable**?

Paradigm 2: Benchmark-Free Validation

Hansen (2026)

Key reframing: an **(LLM + prompt)** pair is a *function*. Treat validation like **goodness-of-fit** in econometrics.

- Annotation: $\hat{\beta} = f^{-1}(y)$ — read text y, return label $\hat{\beta}$.
- Simulation: $\hat{y} = f(\hat{\beta})$ — generate text *from* the label.

$\hat{\beta}$ is **valid** if the regenerated text $\hat{y}$ is *semantically similar* to the original y.

The Regression Analogy

In OLS we estimate $\hat{\beta} = (X'X)^{-1}X'y$ and then ask: do the fitted values $\hat{y} = X\hat{\beta}$ resemble the observed y? Poor fit means the model did not explain the data.

The same logic applies here:

- The **label** $\hat{\beta}$ plays the role of the coefficient — it summarizes what the text is "about."
- The **LLM+prompt** is the data-generating process: given a label, it can produce text of that type.
- **Semantic similarity** between the regenerated and the original text plays the role of goodness-of-fit.

If the regenerated text does not resemble the original, the label failed to capture what the passage is actually about.

Guarding Against Circularity

Using the model to check itself needs two prerequisite properties:

- **Annotation backtranslation**: $f^{-1}(f(\beta)) = \beta$ — re-labeling generated text recovers the label.
- **Separation**: $f(\gamma)$ is *not* semantically similar to $f(\beta)$ for different labels $\gamma \neq \beta$.

Together they rule out a label that "fits" only because the generator is broken in a compensating way.

The Test, Operationally

1. Simulate many passages $\hat{y}$ from the assigned label.

2. Compute the test statistic: cosine similarity (embeddings) between $\hat{y}$ and the original y.

3. Build a null distribution by repetition — **reject** if similarity is no better than chance.

No human labels anywhere — only the model, an embedding model, and the original text.

What It Finds (on Economic Text)

Hansen applies the backtranslation test to several constructs — simple sentiment, granular sentiment (5 levels), clarity, temporal focus, topic mentions — across five passage types: a 10-K filing, an earnings call, a WSJ headline, a Fed governor's speech, a Reddit comment.

For each (construct, passage) pair, the model labels the text and then regenerates a passage from that label. The label is **valid** if the regenerated text is significantly more similar to the original than a random baseline.

- **Simple sentiment, temporal focus, topic mentions**: valid across most passages.
- **Granular sentiment (5 levels), clarity**: often fail — the construct is too fine-grained for the model to regenerate reliably.
- **Larger and newer models** pass more combinations.

The Lesson From Both Papers

- Validity is **task- and passage-specific**. "GPT is accurate" is not a property you can assume — it must be established for *your* construct and *your* corpus.

- Use whichever paradigm fits: a trusted benchmark when you have one, and the backtranslation test when you do not.

- Either way, validation is a step in the research design, not an afterthought.

Beyond Accuracy: Directness

A label can correlate with the truth but be **right for the wrong reasons**:

- **Contamination / look-ahead**: the model "knows" the outcome from pretraining — fatal for forecasting or real-time designs.
- **Shortcut inference**: it guesses the attribute from a correlated cue, not the intended signal.

Accuracy on a benchmark does not catch these — design tests for them too (Asirvatham et al. and Ludwig et al. 2025).

From Disagreement to Measurement Error

Suppose validation gives accuracy 0.90. Then ~10% of labels feeding a regression are **wrong**, and usually not at random.

- As an **outcome**: random error inflates SEs; *systematic* error biases coefficients.
- As a **regressor**: a generated-variable problem — attenuation and worse.

The validation sample is exactly what lets you **correct** it.

The Bridge to Class 5

Both papers land on the same framework (Ludwig et al. 2025):

- An LLM label is a **noisy proxy** for a latent construct.
- Substituting it for the truth attenuates or biases estimands **unless** the error is characterized and corrected — with validation data and the right estimator.

That correction — the **econometrics of generated variables** — is Class 5.

Reproducibility

Open-Weight vs. Closed-API

	Open-weight	Closed API
Exact model frozen?	Yes (pin weights)	No (versions change)
Re-runnable in 5 yrs	Yes, locally	Only if vendor keeps it
Cost / scale	Free, GPU-bound	Pay per call, fast
Quality (hard tasks)	Good	Often better

A modeling *and* a replicability choice. Open-weight models (Llama, Mistral, Falcon) can be pinned to a specific checkpoint and re-run years later; closed APIs can silently change, be deprecated, or alter output distributions between versions.

What Reproducibility Requires in Practice

A paper that cites "GPT-4" is not reproducible. The minimum documentation:

- **Model**: the exact version identifier and access date (e.g. `gpt-4o-2024-08-06`, queried August 2024).
- **Prompt**: the full text, verbatim, in the appendix or replication archive.
- **Parameters**: `temperature = 0` for deterministic output; a `seed` if the API supports it.
- **Output cache**: store the raw model outputs alongside the dataset. If the model changes or the API is retired, you still have the labels you used.
- **Validation sample**: archived with both model and human labels. Without it, no one — including future you — can re-establish that the measure was valid.

Reproducibility Is a Design Choice

Think about reproducibility *before* you run the labels, not after.

- For the most durable option, prefer an **archivable open-weight model** pinned to a specific checkpoint.
- If you use a closed API, document the exact version, prompt, and temperature — and cache the outputs.
- Either way, the validation sample is the most critical artifact to archive: it is the only evidence that your measure was valid at the time of publication.

Optional: Retrieval-Augmented Extraction

The Long-Document Problem

Earnings calls, 10-Ks, and rulings are far longer than a context window, and most of the text is irrelevant to any one question.

Feeding the whole document is costly and dilutes the signal.

RAG: Retrieve, Then Generate

Roesti (2026), "From RAGs to (feature) Riches"

Answer a *specific* question using only the relevant passages:

1. **Embed** the corpus (Class 3).
2. **Retrieve** the chunks most similar to the question.
3. **Prompt** the LLM with just those chunks.

Bounds cost and **grounds** the answer in the source — a feature-extraction pipeline at scale, and a direct payoff from embeddings.

Takeaways

1. Transformers give **contextual** representations; we use them, we don't train them.
2. Pipelines, zero-shot, and prompting turn a corpus into measurements — watch the training-domain mismatch.
3. **Validate every model label**: against a human benchmark, or benchmark-free (Hansen 2026) — and check *directness*, not just accuracy.
4. A model label is a **noisy proxy**; the validation sample is what lets you correct it (Class 5).
5. Reproducibility is a choice — and always archive the validation sample.

Live: Class 4 notebook. **Practice:** Problem Set 4.